

P2413R0

Remove unsafe conversions of `unique_ptr<T>`

Published Proposal, 2021-07-12

Author:

[Lénárd Szolnoki](#) ([Linguamatics](#), an [IQVIA](#) company)

Audience:

SG18 LEWGI, LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

- 1 Motivation
- 2 The core problem
- 3 The proposed solution
- 4 Test results on a large codebase
- 5 Breaking changes
 - 5.1 Destroying delete
- 6 Open questions
- 7 Acknowledgements

§ 1. Motivation

Smart pointers are a success story of modern C++ as they mitigate many dangers of manual memory management. While smart pointers do provide "misusable" named functions (`reset`, `release`), their value-semantic operations (such as assignment and implicit conversions) are generally "easy to use, *impossible* to misuse."

But there is one gap in `unique_ptr`'s safety: sometimes it permits implicit conversions that are actually unsafe. Consider the following program:

```
#include <memory>

struct Base {};
struct Derived : Base {};

int main() {
    std::unique_ptr<Base> base_ptr = std::make_unique<Derived>();
}
```

The delete expression evaluated in the destructor of `base_ptr` deletes an object with dynamic type `Derived` and static type `Base`. As `Base` does not have a virtual destructor the behavior is undefined.

The goal of this proposal is to make these conversions of `unique_ptr` ill-formed. As a result base classes with public non-virtual destructors become safer to use.

§ 2. The core problem

The main problem is the converting constructor of single-object `default_delete`. The converting constructor of `unique_ptr` delegates to the conversion between the deleters of the source and the target types.

Single-object `default_delete` effectively has the following converting constructor template:

```
template <class T>
struct default_delete {
    template <class U>
    requires is_convertible_v<U*, T*>
    default_delete(const default_delete<U>&) noexcept {}
}
```

```

    /* ... */
};

```

Current implementations use SFINAE to express the same constraint.

It is only constrained on the convertibility between the raw pointer types, but it does not consider that the invocation of `operator()` on the resulting object could be undefined.

§ 3. The proposed solution

Assume that `T` and `U` are types so that `is_convertible_v<U*, T*> == true`. Also assume that `u` is a prvalue of type `U*` and `delete u` has defined behavior. I propose to constrain the converting constructor of single-object `default_delete` so that successfully converting `default_delete<U>` to `default_delete<T>` and calling `operator()` on the resulting object with the argument `u` has defined behavior.

Given this constraint and if the destructor call of an object of type `unique_ptr<U>` has defined behavior then a successful conversion of this object to `unique_ptr<T>` maintains this invariant.

```

template <class T>
struct default_delete {
    constexpr default_delete() noexcept = default;

    template<class U>
    requires is_convertible_v<U*, T*>
        && (
            is_similar_v<U, T>
            || has_virtual_destructor_v<T>
        )
    default_delete(const default_delete<U>&) noexcept {}

    /* ... */
};

```

Where `is_similar_v` is an exposition-only type trait to check if two types are similar [\[conv.qual\]](#).

§ 4. Test results on a large codebase

Arthur O'Dwyer made an [LLVM project fork](#) where a similar constraint is applied for the converting constructor of single-object `default_delete` in libc++. This fork was tested against compiling the LLVM project codebase. One of the libc++ tests failed to compile, it originally had undefined behavior (<https://reviews.llvm.org/D90536>). No false positives were found.

§ 5. Breaking changes

§ 5.1. Destroying delete

C++20 introduced destroying `operator delete`. Because of this `delete ptr` might be defined even if the static type of the pointed object does not have a virtual destructor and the static type does not match the dynamic type of the pointed object. Consider the following program:

```
#include <memory>
#include <new>

struct Base {
    void operator delete(Base* ptr, std::destroying_delete_t);
};

struct Derived : Base {};

void Base::operator delete(Base* ptr, std::destroying_delete_t) {
    ::delete static_cast<Derived*>(ptr);
}

int main() {
    std::unique_ptr<Base> base_ptr = std::make_unique<Derived>();
}
```

This program is well-formed in C++20 and has defined behavior, it is however ill-formed with the proposed changes. [P0722R1](#) provides a motivating example with a similar class hierarchy (section "Dynamic dispatch without vptrs"). It is possible that these kind of class hierarchies would be hard to use with `unique_ptr` with the proposed changes. An opt-in customization point to optionally allow the conversion of single-object `default_delete` for certain pair of types could be considered (§ 6 [Open questions](#)).

§ 6. Open questions

1. Should there be a customization point to relax the constraint for class hierarchies involving destroying delete ([§ 5.1 Destroying delete](#)) ?

§ 7. Acknowledgements

I would like to thank Arthur O'Dwyer for his work to test the proposed changes on the LLVM codebase.